

Python for Data Science, AI and Development

January 29, 2024

APPLIED LINGUISTICS GRADUATE PROGRAMME (LAEL)

1 Python for Data Science, AI and Development

1.1 Forms of checking the Python version

```
[1]: # Check the Python version
!python -V
```

Python 3.11.4

```
[2]: # Check the Python version with sys.executable, which is the path to the
      ↪running Python executable
import sys
!{sys.executable} --version
```

Python 3.11.4

```
[3]: # Check the Python version via the sys module
import sys
print('Python version:')
print(sys.version)
print('Python version information:')
print(sys.version_info)
print('Python float_info:')
print(sys.float_info)
```

Python version:

3.11.4 (main, Jul 5 2023, 14:15:25) [GCC 11.2.0]

Python version information:

sys.version_info(major=3, minor=11, micro=4, releaselevel='final', serial=0)

Python float_info:

sys.float_info(max=1.7976931348623157e+308, max_exp=1024, max_10_exp=308,
min=2.2250738585072014e-308, min_exp=-1021, min_10_exp=-307, dig=15,
mant_dig=53, epsilon=2.220446049250313e-16, radix=2, rounds=1)

```
[4]: # Check the Python version via the platform module
from platform import python_version
```

```
print(python_version())
```

3.11.4

1.2 Style - Naming

[Google Python Style Guide - 3.16 Naming](#)

module_name, package_name, ClassName, method_name, ExceptionName, function_name, GLOBAL_CONSTANT_NAME, global_var_name, instance_var_name, function_parameter_name, local_var_name, query_proper_noun_for_thing, send_acronym_via_https.

1.3 Module 1 - Python Basics

1.3.1 Types

```
[5]: type(11)
```

```
[5]: int
```

```
[6]: type(21.213)
```

```
[6]: float
```

```
[7]: type('Hello Python!')
```

```
[7]: str
```

Type casting

```
[8]: float(2)
```

```
[8]: 2.0
```

```
[9]: type(float(2))
```

```
[9]: float
```

```
[10]: int(1.1)
```

```
[10]: 1
```

```
[11]: int('1')
```

```
[11]: 1
```

```
[12]: # The following line is inconsistent and generates an error
      # int('A')
```

```
[13]: str(1)
```

```
[13]: '1'
```

```
[14]: str(4.5)
```

```
[14]: '4.5'
```

```
[15]: type(True)
```

```
[15]: bool
```

```
[16]: type(False)
```

```
[16]: bool
```

```
[17]: bool(1)
```

```
[17]: True
```

```
[18]: bool(False)
```

```
[18]: False
```

```
[19]: int(True)
```

```
[19]: 1
```

```
[20]: int(False)
```

```
[20]: 0
```

```
[21]: type(bool(1))
```

```
[21]: bool
```

```
[22]: float(True)
```

```
[22]: 1.0
```

1.3.2 Expressions and Variables

Expressions

```
[23]: 43 + 60 + 16 + 41
```

```
[23]: 160
```

```
[24]: 25 / 6
```

```
[24]: 4.166666666666667
```

```
[25]: 25 // 6
```

```
[25]: 4
```

```
[26]: 11 // 2
```

```
[26]: 5
```

```
[27]: (30 + 2) * 60
```

```
[27]: 1920
```

```
[28]: type(6 / 2)
```

```
[28]: float
```

```
[29]: myvar = 1 / 1  
      type(myvar)
```

```
[29]: float
```

```
[30]: type(int(6 / 2))
```

```
[30]: int
```

```
[31]: type(6//2)
```

```
[31]: int
```

Variables

```
[32]: my_variable = 15  
      print(my_variable)
```

```
15
```

```
[33]: my_variable
```

```
[33]: 15
```

```
[34]: x = 43 + 60 + 16 + 41  
      print(x)
```

```
160
```

```
[35]: y = x / 60  
      print(y)
```

2.6666666666666665

```
[36]: z = 160  
      z = z / 60  
      print(z)
```

2.6666666666666665

```
[37]: w = 4  
      w = w / 2  
      print(w)
```

2.0

```
[38]: total_min = 43 + 42 + 57  
      total_hr = total_min / 60  
      print(total_hr)
```

2.3666666666666667

1.3.3 String Operations

```
[39]: len('Michael Jackson')
```

[39]: 15

```
[40]: Name = 'Michael Jackson'  
      Sentence = Name + ' is the best'  
      print(Sentence)
```

Michael Jackson is the best

```
[41]: print(Name[0])
```

M

```
[42]: print(Name[-1])
```

n

Note: Python supports two forms of indexing. Notice the differences: - Positive indexing: 0 | 1 | 2 | 3 - Negative indexing: -4 | -3 | -2 | -1

```
[43]: Name[0:4]
```

[43]: 'Mich'

Note: In Python, when performing slicing, the last index is one larger than the element we want.

```
[44]: Name[8:12]
```

```
[44]: 'Jack'
```

```
[45]: Name[:2]
```

```
[45]: 'McalJcsn'
```

```
[46]: Name[0:5:2]
```

```
[46]: 'Mca'
```

```
[47]: 3 * 'Michael Jackson '
```

```
[47]: 'Michael Jackson Michael Jackson Michael Jackson '
```

```
[48]: print('Michael Jackson\nis the best')
```

```
Michael Jackson
is the best
```

```
[49]: print('Michael Jackson \t is the best')
```

```
Michael Jackson      is the best
```

```
[50]: print('Michael Jackson \\ is the best')
```

```
Michael Jackson \ is the best
```

```
[51]: print(r'Michael Jackson \ is the best')
```

```
Michael Jackson \ is the best
```

```
[52]: A = 'Thriller is the sixth studio album'
      print('Before upper:', A)
      B = A.upper()
      print('After upper:', B)
```

```
Before upper: Thriller is the sixth studio album
After upper: THRILLER IS THE SIXTH STUDIO ALBUM
```

```
[53]: A = 'Michael Jackson is the best'
      B = A.replace('Michael', 'Janet')
      print(B)
```

```
Janet Jackson is the best
```

```
[54]: Name = 'Michael Jackson'
      print(Name.find('el'))
```

5

```
[55]: print(Name.find('ah'))
```

-1

```
[56]: print('hello Mike'.find('Mike'))
```

6

1.4 Module 2 - Python Data Structures

1.4.1 Lists and Tuples

Also known as compound data types.

Tuples

- Tuples are an ordered sequence
- Tuples are written as comma-separated elements within **parentheses**
- **Tuples are immutable**

```
[57]: Ratings = (10, 9, 6, 5, 10, 8, 9, 6, 2)
      print(Ratings[0:4])
      type(Ratings)
```

(10, 9, 6, 5)

```
[57]: tuple
```

```
[58]: Ratings2 = sorted(Ratings)
      print(Ratings2)
```

[2, 5, 6, 6, 8, 9, 9, 10, 10]

```
[59]: say_what = ('say', 'what', 'you', 'will')
      print(say_what[-1])
```

will

```
[60]: A = (1, 2, 3, 4, 5)
      print(A[1:4])
```

(2, 3, 4)

```
[61]: A = (1, 2, 3, 4, 5)
      len(A)
```

```
[61]: 5
```

```
[62]: Nesting = (1, 2, ('pop', 'rock'), (3, 4), ('disco', (1, 2)))
print(Nesting[2][1])
print(Nesting[2][1][0])
print(Nesting[2][1][1])
```

```
rock
r
o
```

```
[63]: print(Nesting.index((3, 4)))
```

```
3
```

Lists

- Lists are also ordered sequences
- Lists are written as comma-separated elements within **brackets**
- **Lists are mutable**

```
[64]: L = ['Michael Jackson', 10.1, 1982, [1, 2], ('A', 1)]
```

```
[65]: B = [1, 2, [3, 'a'], [4, 'b']]
print(B[3][1])
```

```
b
```

Lists can be concatenated:

```
[66]: List1 = [1, 2, 3]
List2 = [1, 1, 1]
print(List1 + List2)
```

```
[1, 2, 3, 1, 1, 1]
```

```
[67]: print([1, 2, 3] + [1, 1, 1])
```

```
[1, 2, 3, 1, 1, 1]
```

Lists are mutable, they can be changed. Everytime we apply a method the list changes:

```
[68]: L = ['Michael Jackson', 10.1, 1982]
print(L)
```

```
['Michael Jackson', 10.1, 1982]
```

```
[69]: L.extend(['pop', 10])
print(L)
```

```
['Michael Jackson', 10.1, 1982, 'pop', 10]
```



```
[70]: # The method 'append' adds one element to a list
```

```
L.append(['flamboyant', 234])  
print(L)
```

```
['Michael Jackson', 10.1, 1982, 'pop', 10, ['flamboyant', 234]]
```

```
[71]: A = [1]  
A.append([2, 3, 4, 5])  
print(A)  
len(A)
```

```
[1, [2, 3, 4, 5]]
```

```
[71]: 2
```

```
[72]: A = ['Disco', 10, 1.2]  
print(A)  
A[0] = 'Hard rock'  
print(A)  
del(A[2])  
print(A)
```

```
['Disco', 10, 1.2]  
['Hard rock', 10, 1.2]  
['Hard rock', 10]
```

```
[73]: 'Hello Mike'.split()
```

```
[73]: ['Hello', 'Mike']
```

```
[74]: 'A,B,C,D'.split(',')
```

```
[74]: ['A', 'B', 'C', 'D']
```

Aliasing: Both A and B are referencing the same list

```
[75]: A = ['Hard rock', 10, 1.2]  
B = A  
print(A)  
print(B)  
del(A[2])  
print(A)  
print(B)
```

```
['Hard rock', 10, 1.2]  
['Hard rock', 10, 1.2]
```

```
['Hard rock', 10]
['Hard rock', 10]
```

Cloning: B becomes a clone of A

```
[76]: A = ['Hard rock', 10, 1.2]
      B = A[:]
      print(A)
      print(B)
      del(A[2])
      print(A)
      print(B)
```

```
['Hard rock', 10, 1.2]
['Hard rock', 10, 1.2]
['Hard rock', 10]
['Hard rock', 10, 1.2]
```

Getting help

```
[77]: A = ['Hard rock', 10, 1.2]
      help(A)
```

Help on list object:

```
class list(object)
| list(iterable=(), /)
|
| Built-in mutable sequence.
|
| If no argument is given, the constructor creates a new empty list.
| The argument must be an iterable if specified.
|
| Methods defined here:
|
| __add__(self, value, /)
|     Return self+value.
|
| __contains__(self, key, /)
|     Return key in self.
|
| __delitem__(self, key, /)
|     Delete self[key].
|
| __eq__(self, value, /)
|     Return self==value.
|
| __ge__(self, value, /)
|     Return self>=value.
```

```

|  __getattr__(self, name, /)
|      Return getattr(self, name).
|
|  __getitem__(...)
|      x.__getitem__(y) <==> x[y]
|
|  __gt__(self, value, /)
|      Return self>value.
|
|  __iadd__(self, value, /)
|      Implement self+=value.
|
|  __imul__(self, value, /)
|      Implement self*=value.
|
|  __init__(self, /, *args, **kwargs)
|      Initialize self. See help(type(self)) for accurate signature.
|
|  __iter__(self, /)
|      Implement iter(self).
|
|  __le__(self, value, /)
|      Return self<=value.
|
|  __len__(self, /)
|      Return len(self).
|
|  __lt__(self, value, /)
|      Return self<value.
|
|  __mul__(self, value, /)
|      Return self*value.
|
|  __ne__(self, value, /)
|      Return self!=value.
|
|  __repr__(self, /)
|      Return repr(self).
|
|  __reversed__(self, /)
|      Return a reverse iterator over the list.
|
|  __rmul__(self, value, /)
|      Return value*self.
|
|  __setitem__(self, key, value, /)
|      Set self[key] to value.

```

```

|
|  __sizeof__(self, /)
|      Return the size of the list in memory, in bytes.
|
|  append(self, object, /)
|      Append object to the end of the list.
|
|  clear(self, /)
|      Remove all items from list.
|
|  copy(self, /)
|      Return a shallow copy of the list.
|
|  count(self, value, /)
|      Return number of occurrences of value.
|
|  extend(self, iterable, /)
|      Extend list by appending elements from the iterable.
|
|  index(self, value, start=0, stop=9223372036854775807, /)
|      Return first index of value.
|
|      Raises ValueError if the value is not present.
|
|  insert(self, index, object, /)
|      Insert object before index.
|
|  pop(self, index=-1, /)
|      Remove and return item at index (default last).
|
|      Raises IndexError if list is empty or index is out of range.
|
|  remove(self, value, /)
|      Remove first occurrence of value.
|
|      Raises ValueError if the value is not present.
|
|  reverse(self, /)
|      Reverse *IN PLACE*.
|
|  sort(self, /, *, key=None, reverse=False)
|      Sort the list in ascending order and return None.
|
|      The sort is in-place (i.e. the list itself is modified) and stable (i.e.
the
|      order of two equal elements is maintained).
|
|      If a key function is given, apply it once to each list item and sort

```

```

them,
|     ascending or descending, according to their function values.
|
|     The reverse flag can be set to sort in descending order.
|
|     -----
|     Class methods defined here:
|
|     __class_getitem__(...) from builtins.type
|         See PEP 585
|
|     -----
|     Static methods defined here:
|
|     __new__(*args, **kwargs) from builtins.type
|         Create and return a new object.  See help(type) for accurate signature.
|
|     -----
|     Data and other attributes defined here:
|
|     __hash__ = None

```

1.4.2 Dictionaries

- Dictionaries are denoted with **curly brackets**
- The keys have to be immutable and unique
- The values can be immutable, mutable and duplicates
- Each key is separated from its value by a colon
- Each key and value pair is separated by a comma

```

[78]: Dict = {'Thriller': 1982, 'Back in Black': 1980, 'The Bodyguard': 1992}
print(Dict['The Bodyguard'])

```

1992

```

[79]: Dict2 = {'A':1, 'B':2, 'C':[3,3,3], 'D':(4,4,4), 'E':5, 'F':6}
print(Dict2['D'])

```

(4, 4, 4)

```

[80]: Dict['Graduation'] = 2007
print(Dict)

```

```

{'Thriller': 1982, 'Back in Black': 1980, 'The Bodyguard': 1992, 'Graduation': 2007}

```

```
[81]: del(Dict['Thriller'])
      print(Dict)
      Dict['Thriller'] = 1982
      print(Dict)
```

```
{'Back in Black': 1980, 'The Bodyguard': 1992, 'Graduation': 2007}
{'Back in Black': 1980, 'The Bodyguard': 1992, 'Graduation': 2007, 'Thriller': 1982}
```

```
[82]: 'The Bodyguard' in Dict
```

```
[82]: True
```

```
[83]: 'Starboy' in Dict
```

```
[83]: False
```

```
[84]: print(Dict.keys())
```

```
dict_keys(['Back in Black', 'The Bodyguard', 'Graduation', 'Thriller'])
```

```
[85]: print(Dict.values())
```

```
dict_values([1980, 1992, 2007, 1982])
```

1.4.3 Sets

- Sets are a type of collection
 - This means that like lists and tuples you can input different Python types
- Unlike lists and tuples they are unordered
 - This means sets do not record element position
- Sets only have **unique elements**
 - This means there is only one of a particular element in a set

```
[86]: album_list = ['Michael Jackson', 'Thriller', 'Thriller', 1982]
      print(album_list)
      album_set = set(album_list)
      print(album_set)
      type(album_set)
```

```
['Michael Jackson', 'Thriller', 'Thriller', 1982]
{'Michael Jackson', 'Thriller', 1982}
```

```
[86]: set
```

```
[87]: album_set.add('NSYNC')
      print(album_set)
```

```
{'Michael Jackson', 'Thriller', 'NSYNC', 1982}
```

```
[88]: # If we try to add an element that already existis, nothing will happen because  
↪the elements of a set have to be unique  
album_set.add('NSYNC')  
print(album_set)
```

```
{'Michael Jackson', 'Thriller', 'NSYNC', 1982}
```

```
[89]: album_set.remove('NSYNC')  
print(album_set)
```

```
{'Michael Jackson', 'Thriller', 1982}
```

```
[90]: print('Thriller' in album_set)  
print('Cindy Lauper' in album_set)
```

```
True  
False
```

```
[91]: album_set2 = {'Joe Cocker', 'Michael Jackson', 1972, 'The Dark Side of the  
↪Moon', 1982}  
print(album_set2)  
print(album_set)  
album_set3 = album_set & album_set2  
print(album_set3)
```

```
{'Joe Cocker', 1972, 'The Dark Side of the Moon', 'Michael Jackson', 1982}  
{'Michael Jackson', 'Thriller', 1982}  
{'Michael Jackson', 1982}
```

```
[92]: print(album_set.intersection(album_set2))
```

```
{'Michael Jackson', 1982}
```

```
[93]: print(album_set2)  
print(album_set)  
album_set4 = album_set.union(album_set2)  
print(album_set4)
```

```
{'Joe Cocker', 1972, 'The Dark Side of the Moon', 'Michael Jackson', 1982}  
{'Michael Jackson', 'Thriller', 1982}  
{'Joe Cocker', 'Thriller', 1972, 'The Dark Side of the Moon', 'Michael Jackson',  
1982}
```

```
[94]: print(album_set.issubset(album_set2))  
print(album_set.issubset(album_set4))
```

```
False  
True
```

```
[95]: print(album_set4.issuperset(album_set))
```

True

```
[96]: print(album_set4.difference(album_set))
```

{'Joe Cocker', 1972, 'The Dark Side of the Moon'}

1.5 Module 3 - Python Programming Fundamentals

1.5.1 Conditions and Branching

Conditions

```
[97]: # Equality operator
```

```
a = 6
print(a == 7)
print(a == 6)
```

False

True

```
[98]: # Greater to operator
```

```
b = 7
print(b > 7)
print(b >= 7)
print(b > 6)
```

False

True

True

```
[99]: # Less than operator
```

```
c = 7
print(c < 7)
print(c <= 7)
print(c < 8)
```

False

True

True

```
[100]: # Inequality operator
```

```
d = 6
print(d != 5)
print(d != 6)
```



```
print(d != 7)
```

True
False
True

Branching

[101]: *# The if statement*

```
age = 20
if (age > 18):
    print('you can enter')
print('move on')
age = 17
if (age > 18):
    print('you can enter')
print('move on')
```

you can enter
move on
move on

[102]: *# The else statement*

```
age = 20
if (age > 18):
    print('you can enter')
else:
    print('go see Meat Loaf')
print('move on')
age = 17
if (age > 18):
    print('you can enter')
else:
    print('go see Meat Loaf')
print('move on')
```

you can enter
move on
go see Meat Loaf
move on

[103]: *# The elif statement*

```
age = 20
if (age > 18):
    print('you can enter')
```

```

elif (age == 18):
    print('go see Pink Floyd')
else:
    print('go see Meat Loaf')
print('move on')
age = 18
if (age > 18):
    print('you can enter')
elif (age == 18):
    print('go see Pink Floyd')
else:
    print('go see Meat Loaf')
print('move on')
age = 17
if (age > 18):
    print('you can enter')
elif (age == 18):
    print('go see Pink Floyd')
else:
    print('go see Meat Loaf')
print('move on')

```

```

you can enter
move on
go see Pink Floyd
move on
go see Meat Loaf
move on

```

Logic operators

[104]: *# The not operator*

```

print(not(True))
print(not(False))

```

```

False
True

```

[105]: *# The or operator*

```

album_year = 1990
if (album_year < 1980) or (album_year > 1989):
    print("The album was made in the 70's or 90's")
else:
    print("The album was made in the 1980's")
album_year = 1985
if (album_year < 1980) or (album_year > 1989):

```

```

    print("The album was made in the 70's or 90's")
else:
    print("The album was made in the 1980's")
album_year = 1995
if (album_year < 1980) or (album_year > 1989):
    print("The album was made in the 70's or 90's")
else:
    print("The album was made in the 1980's")

```

The album was made in the 70's or 90's
The album was made in the 1980's
The album was made in the 70's or 90's

```

[106]: # The and operator

album_year = 1983
if (album_year > 1979) and (album_year < 1990):
    print("This album was made in the 80's")

```

This album was made in the 80's

There are 2 sisters, Annie and Jane, born in 1996 and 1999 respectively. They want to know who was born in a leap year. Write an if-else statement to determine who was born in a leap year. [Definition of leap year](#)

```

[107]: # Solution 1

year = 2000
if (year / 100 - year // 100) == 0 and (year / 400 - year // 400) == 0:
    print('You were born in a end-of-century leap year')
elif (year / 4 - year // 4) == 0:
    print('You were born in a leap year')
else:
    print('You were not born in a leap year')

```

You were born in a end-of-century leap year

```

[108]: # Solution 2 (using the module operator)

year = 1996
if (year % 100) == 0 and (year % 400) == 0:
    print('You were born in a end-of-century leap year')
elif (year % 4) == 0:
    print('You were born in a leap year')
else:
    print('You were not born in a leap year')

```

You were born in a leap year

In a school canteen, children under the age of 9 are only given milk porridge for breakfast. Children from 10 to 14 are given a sandwich, and children from 15 to 17 are given a burger. The canteen master asks the age of the student and gives them breakfast accordingly. Sam's age is 10. Use if-else statement to determine what the canteen master will offer to him.

```
[109]: Sams_age = 10
if Sams_age <= 9:
    print('The canteen master will offer him porridge')
elif Sams_age >= 10 and Sams_age <= 14:
    print('The canteen master will offer him a sandwich')
elif Sams_age >= 15 and Sams_age <= 17:
    print('The canteen master will offer him a burger')
elif Sams_age >= 18:
    print('The canteen master will not know what to offer him')
```

The canteen master will offer him a sandwich

1.5.2 Loops

For loops

```
[110]: # The range function returns a range object
print(range(5))
print(range(10, 20))
```

```
range(0, 5)
range(10, 20)
```

```
[111]: # for loops are used when a certain piece of code has to be iterated for a
       ↪ predefined (known) number of times
for x in range(0, 3):
    print(x)
```

```
0
1
2
```

```
[112]: for y in range(15, 20):
        print(y)
```

```
15
16
17
18
19
```

```
[113]: for x in ['A', 'B', 'C']:
        print(x+'A')
```

AA
BA
CA

```
[114]: # When using indexes, you have to manage the indexes and the elements of the
        ↪ list being referenced to separately
squares = ['red', 'yellow', 'green', 'purple', 'blue']
for i in range(0, 5):
    print(squares[i])
```

red
yellow
green
purple
blue

```
[115]: # Using the len function to determine the extent of the range
squares = ['red', 'yellow', 'green', 'purple', 'blue']
for i in range(0, len(squares)):
    print(squares[i])
```

red
yellow
green
purple
blue

```
[116]: # Not using indexes, directly referencing the elements of a list
squares = ['red', 'yellow', 'green', 'purple', 'blue']
for square in squares:
    print(square)
```

red
yellow
green
purple
blue

```
[117]: # Using the enumerate function
squares = ['red', 'yellow', 'green', 'purple', 'blue']
for i, square in enumerate(squares):
    print(i, square)
```

0 red
1 yellow
2 green
3 purple
4 blue

While loops

[118]: *# while loops are used when a certain piece of code has to be iterated until a
↪ certain condition is met. It involves the initialisation of variables that
↪ control the condition being tested*

```
count = 1
while count <= 5:
    print(count)
    count += 1
```

1
2
3
4
5

```
[119]: squares = ['orange', 'orange', 'green', 'orange', 'blue']
newsquares = []
i = 0
while(squares[i] == 'orange'):
    newsquares.append(squares[i])
    i = i+1
    print(newsquares)
```

['orange']
['orange', 'orange']

The following is a list of animals in a National Zoo. Animals = ["lion", "giraffe", "gorilla", "parrots", "crocodile", "deer", "swan"]

Your brother needs to write an essay on the animals whose names are made of 7 letters. Help him find those animals through a while loop and create a separate list of such animals.

```
[120]: Animals = ['lion', 'giraffe', 'gorilla', 'parrots', 'crocodile', 'deer', 'swan']
SevenLetterList = []
i = 0
print('7-letter list of animals:')
while i < len(Animals):
    if len(Animals[i]) == 7:
        SevenLetterList.append(Animals[i])
    i += 1
print(SevenLetterList)
```

7-letter list of animals:
['giraffe', 'gorilla', 'parrots']

1.5.3 Functions

```
[121]: # Sorted function. A function does not change the input list
album_ratings = [10.0, 8.5, 9.5, 7.0, 7.0, 9.5, 9.0, 9.5]
sorted_album_ratings = sorted(album_ratings)
print(sorted_album_ratings)
print(album_ratings)
```

```
[7.0, 7.0, 8.5, 9.0, 9.5, 9.5, 9.5, 10.0]
[10.0, 8.5, 9.5, 7.0, 7.0, 9.5, 9.0, 9.5]
```

```
[122]: # Sort method. A method changes the input list
album_ratings = [10.0, 8.5, 9.5, 7.0, 7.0, 9.5, 9.0, 9.5]
print(album_ratings)
album_ratings.sort()
print(album_ratings)
```

```
[10.0, 8.5, 9.5, 7.0, 7.0, 9.5, 9.0, 9.5]
[7.0, 7.0, 8.5, 9.0, 9.5, 9.5, 9.5, 10.0]
```

```
[123]: def add1(a):
        b = a + 1
        return(b)
print(add1(5))
```

6

```
[124]: # Use triple quotes (docstring) to document what the fuction does
def add1(a):
    """
    add 1 to a
    """
    b = a + 1
    return(b)
print(add1(5))
help(add1)
```

6

Help on function add1 in module __main__:

```
add1(a)
    add 1 to a
```

```
[125]: def mult(a, b):
        '''
        multiply a and b
        '''
```

```

    c = a * b
    return(c)
print(mult(2, 3))
help(mult)

```

6

Help on function mult in module __main__:

```

mult(a, b)
    multiply a and b

```

```

[126]: def mult(a, b):
        '''
        multiply a and b
        '''
        c=a*b
        return(c)
print(mult(2, 'Michael Jackson '))
help(mult)

```

Michael Jackson Michael Jackson

Help on function mult in module __main__:

```

mult(a, b)
    multiply a and b

```

```

[127]: # If a function does not have a return statement, it returns a special None
        ↪object
def MJ():
    print('Michael Jackson')
print(MJ())

```

Michael Jackson

None

```

[128]: # A function cannot have an empty body so we use the keyword pass
def NoWork():
    pass
print(NoWork())

```

None

```

[129]: # It is interesting to see the function NoWork with the return statement None
def NoWork():
    pass
    return None

```



```
print(NoWork())
```

None

```
[130]: def add1(a):  
        b = a + 1;  
        print(a, 'plus 1 equals', b)  
        return b  
add1(2)
```

2 plus 1 equals 3

[130]: 3

```
[131]: def printStuff(Stuff):  
        for i, s in enumerate(Stuff):  
            print('Album', i, 'rating is ', s)  
album_ratings = [10.0, 8.5, 9.5]  
printStuff(album_ratings)
```

Album 0 rating is 10.0

Album 1 rating is 8.5

Album 2 rating is 9.5

```
[132]: def Print(A):  
        for a in A:  
            print(a + '1')  
Print(['a', 'b', 'c'])
```

a1

b1

c1

```
[133]: # Collecting arguments  
def ArtistNames(*names):  
    for name in names:  
        print(name)  
ArtistNames('Michael Jackson', 'AC/DC', 'Pink Floyd')
```

Michael Jackson

AC/DC

Pink Floyd

```
[134]: # Collecting arguments  
def ArtistNames(*names):  
    for name in names:  
        print(name)  
ArtistNames('Michael Jackson', 'AC/DC')
```

Michael Jackson
AC/DC

```
[135]: # Variables within a global or local scope. Variables inside the global scope
        ↪ can have the same names as variables in the local scope with no conflict
def Thriller():
    Date = 1982
    return(Date)
Date = 2017
print(Thriller())
print(Date)
```

1982
2017

```
[136]: # When a variable is not defined in the local scope, Python will leave the
        ↪ scope and check if the variable exists in the global scope
def ACDC(y):
    print(Rating)
    return(Rating + y)
Rating = 9
Z=ACDC(1)
print(Z)
```

9
10

```
[137]: # Using the keyword global to create a global variable inside a function
def PinkFloyd():
    global ClaimedSales
    ClaimedSales = '45 million'
    return ClaimedSales
PinkFloyd()
print(ClaimedSales)
```

45 million

Count the Frequency of Words Appearing in a String Using a Dictionary.

Find the count of occurrence of any word in our string using python. This is what we are going to do in this section, count the number of word in a given string and print it.

Lets suppose we have a 'string' and the 'word' and we need to find the count of occurrence of this word in our string using python. This is what we are going to do in this section, count the number of word in a given string and print it.

The first thing, we will do is define a function and then create a list that will be empty initially.

Next, we will add a code to convert the string to a list. Python string has a split() method. It takes a string and some separator to return a list.

Now we will declare an empty dictionary.

Next we will add code using for loop to iterate the words and value will count the frequency of each words in the string and store them to the dictionary.

Finally we will print the dictionary.

```
[138]: def countWords(string):  
        words = []  
        words = string.split()  
        dictionary = {}  
        for key in words:  
            dictionary[key] = words.count(key)  
        return(dictionary)  
sentence = 'Mary had a little lamb Little lamb, little lamb Mary had a little,  
↳lamb.Its fleece was white as snow And everywhere that Mary went Mary went,↳  
↳Mary went \\  
Everywhere that Mary went The lamb was sure to go'  
print(countWords(sentence))
```

```
{'Mary': 6, 'had': 2, 'a': 2, 'little': 3, 'lamb': 3, 'Little': 1, 'lamb,': 1,  
'lamb.Its': 1, 'fleece': 1, 'was': 2, 'white': 1, 'as': 1, 'snow': 1, 'And': 1,  
'everywhere': 1, 'that': 2, 'went': 3, 'went,': 1, 'Everywhere': 1, 'The': 1,  
'sure': 1, 'to': 1, 'go': 1}
```

1.5.4 Exception Handling

```
[139]: a = 1  
try:  
    b = int(input('Please enter a number to divide a'))  
    a = a/b  
    print('Success a=',a)  
except:  
    print('There was an error')
```

Please enter a number to divide a b

There was an error

```
[140]: a = 1  
  
try:  
    b = int(input('Please enter a number to divide a'))  
    a = a/b  
    print('Success a=',a)  
except ZeroDivisionError:  
    print('The number you provided cant divide 1 because it is 0')  
except ValueError:  
    print('You did not provide a number')
```

```
except:
    print('Something went wrong')
```

Please enter a number to divide a b

You did not provide a number

```
[141]: a = 1
try:
    b = int(input('Please enter a number to divide a'))
    a = a/b
except ZeroDivisionError:
    print('The number you provided cant divide 1 because it is 0')
except ValueError:
    print('You did not provide a number')
except:
    print('Something went wrong')
else:
    print('success a=',a)
finally:
    print('Processing Complete')
```

Please enter a number to divide a 5

success a= 0.2

Processing Complete

1.5.5 Objects and Classes

Methods

- A class or type's methods are **functions that every instance of that class or type provides**
- It's **how you change or interact with the data in an object**
- *Sorting* is an example of a method that interacts with the data in the object

```
[142]: ratings = [10, 9, 6 ,5, 10, 8, 9, 6, 2]
print(ratings)
ratings.sort()
print(ratings)
ratings.reverse()
print(ratings)
```

```
[10, 9, 6, 5, 10, 8, 9, 6, 2]
```

```
[2, 5, 6, 6, 8, 9, 9, 10, 10]
```

```
[10, 10, 9, 9, 8, 6, 6, 5, 2]
```

```
[143]: # Import the matplotlib library
import matplotlib.pyplot as plt
%matplotlib inline
```

Class Circle

```
[144]: # Create a class circle
class Circle(object):
    def __init__(self, radius = 3, colour = 'blue'):
        self.radius = radius
        self.colour = colour
    def add_radius(self, r):
        self.radius = self.radius + r
        return(self.radius)
    def drawCircle(self):
        plt.gca().add_patch(plt.Circle((0, 0), radius = self.radius, fc = self.
↪colour))
        plt.axis('scaled')
        plt.show()
```

```
[145]: # Create an object RedCircle
RedCircle = Circle(10, 'red')
```

```
[146]: dir(RedCircle)
```

```
[146]: ['__class__',
        '__delattr__',
        '__dict__',
        '__dir__',
        '__doc__',
        '__eq__',
        '__format__',
        '__ge__',
        '__getattr__',
        '__getstate__',
        '__gt__',
        '__hash__',
        '__init__',
        '__init_subclass__',
        '__le__',
        '__lt__',
        '__module__',
        '__ne__',
        '__new__',
        '__reduce__',
        '__reduce_ex__',
        '__repr__',
        '__setattr__',
        '__sizeof__',
        '__str__',
        '__subclasshook__',
        '__weakref__']
```

```
'add_radius',  
'colour',  
'drawCircle',  
'radius']
```

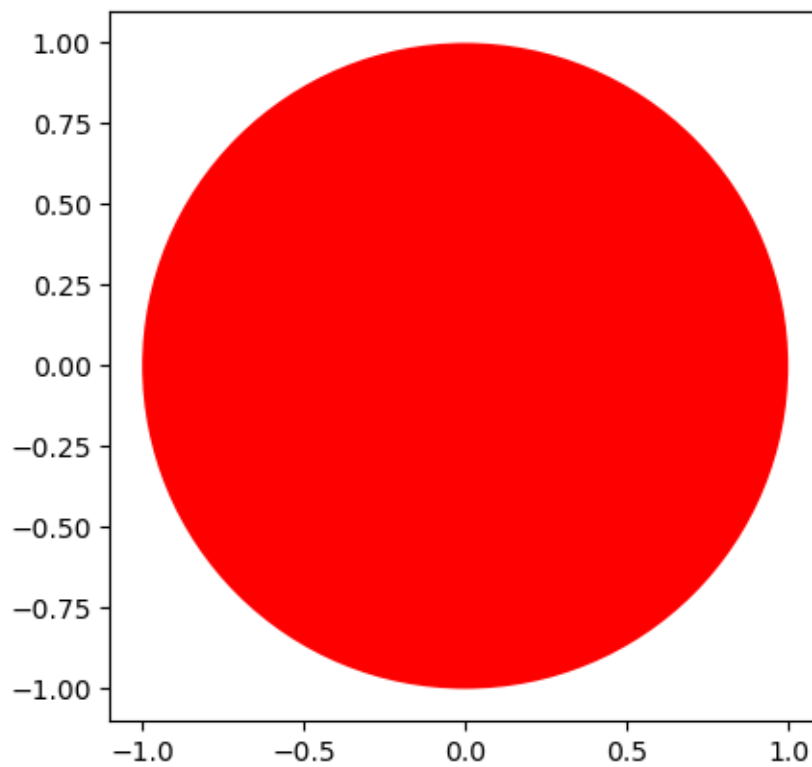
```
[147]: print(RedCircle.radius)  
       print(RedCircle.colour)
```

```
10  
red
```

```
[148]: RedCircle.radius = 1  
       print(RedCircle.radius)
```

```
1
```

```
[149]: RedCircle.drawCircle()
```

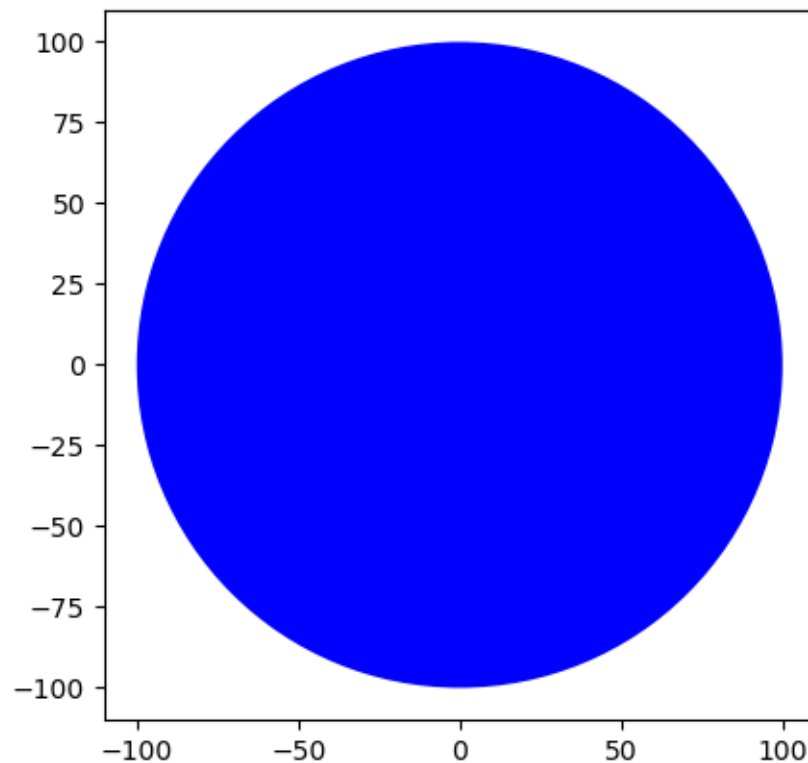


```
[150]: RedCircle.add_radius(2)  
       print(RedCircle.radius)  
       RedCircle.add_radius(5)  
       print(RedCircle.radius)
```

3
8

```
[151]: BlueCircle = Circle(radius = 100)
print(BlueCircle.radius)
print(BlueCircle.colour)
BlueCircle.drawCircle()
```

100
blue



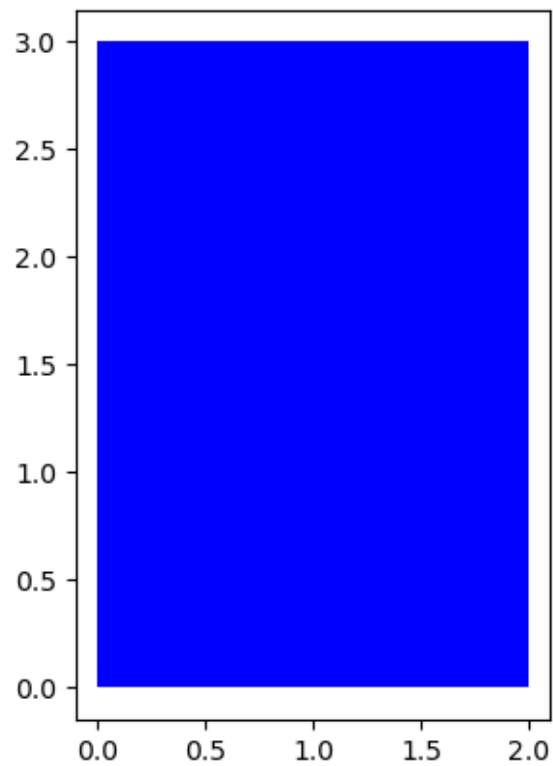
Class Rectangle

```
[152]: class Rectangle(object):
    def __init__(self, width = 2, height = 3, colour = 'red'):
        self.width = width
        self.height = height
        self.colour = colour
    def drawRectangle(self):
        plt.gca().add_patch(plt.Rectangle((0, 0), self.width, self.height,
↪,fc=self.colour))
        plt.axis('scaled')
        plt.show()
```

```
[153]: # Create an object SkinnyBlueRectangle
SkinnyBlueRectangle = Rectangle(2, 3, 'blue')
print(SkinnyBlueRectangle.width)
print(SkinnyBlueRectangle.height)
print(SkinnyBlueRectangle.colour)
```

```
2
3
blue
```

```
[154]: SkinnyBlueRectangle.drawRectangle()
```

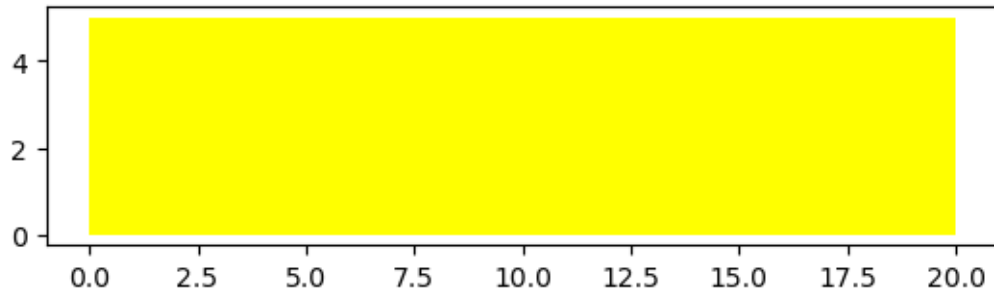


```
[155]: # Create an object FatYellowRectangle
FatYellowRectangle = Rectangle(20, 5, 'yellow')
print(FatYellowRectangle.width)
print(FatYellowRectangle.height)
print(FatYellowRectangle.colour)
```

```
20
5
yellow
```



```
[156]: FatYellowRectangle.drawRectangle()
```

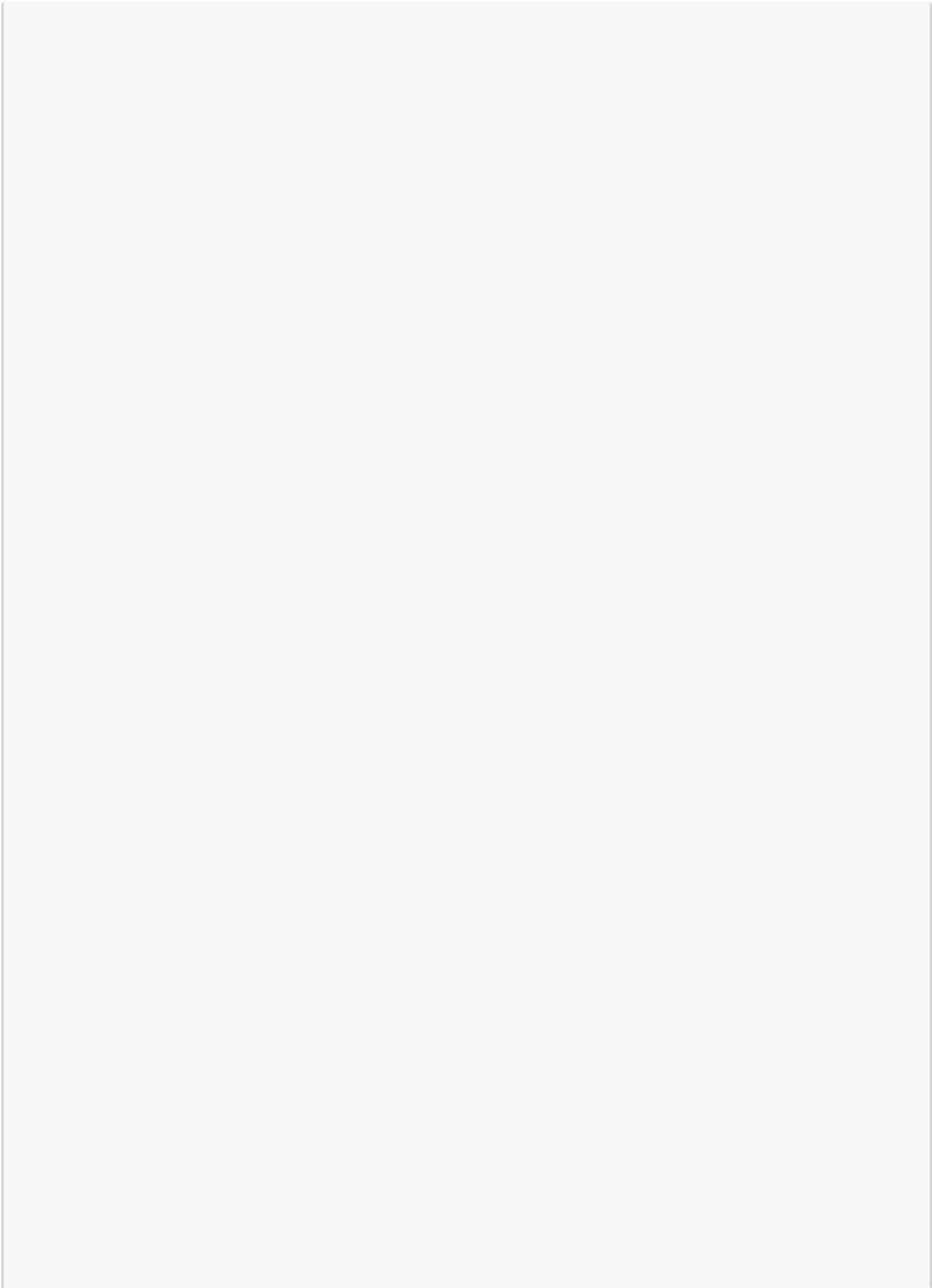


1.5.6 Practice Lab

Text Analysis Text analysis, also known as text mining or text analytics, refers to the process of extracting meaningful information and insights from textual data

```
[157]: class textAnalyser(object):
        """
        Provides methods for basic text analysis
        """
        def __init__(self, text):
            """
            Remove punctuation convert the characters to lower case
            """
            bareText = text.replace('.', '').replace('!', '').replace('?', '').
↪replace(',', ' ')
            bareText = bareText.lower()
            self.bText = bareText
        def freqAll(self):
            """
            Creates a dictionary with word counts
            """
            wordList = self.bText.split(' ')
            freqDic = {}
            for word in set(wordList):
                freqDic[word] = wordList.count(word)
            return(freqDic)
        def freqOf(self, word):
            """
            Returns the frequency of a given word
            """
            freqMap = self.freqAll()
            if word in freqMap:
                return freqMap[word]
```

```
    else:
        return(0)
# inputText = 'Lorem ipsum dolor! diam amet, consetetur Lorem magna. sed diam
↳ nonummy eirmod tempor. diam et labore? et diam magna. et diam amet.'
# I used used Katherine Mansfield's 'Miss Brill' short story but I had to
↳ remove the special character '-' and the string delimiters ' ' and ''
```



```

analysedText = textAnalyser(inputText)
print('Bare text: ', analysedText.bText)
freqWords = analysedText.freqAll()
print(freqWords)
# word = 'lorem'
word = 'ermine'
frequencyOf = analysedText.freqOf(word)
print('The word', word, 'appears', frequencyOf, 'times.')

```

Bare text: miss brill although it was so brilliantly fine-the blue sky powdered with gold and great spots of light like white wine splashed over the jardins publiques-miss brill was glad that she had decided on her fur the air was motionless but when you opened your mouth there was just a faint chill like a chill from a glass of iced water before you sip and now and again a leaf came drifting-from nowhere from the sky miss brill put up her hand and touched her fur dear little thing it was nice to feel it again she had taken it out of its box that afternoon shaken out the moth-powder given it a good brush and rubbed the life back into the dim little eyes what has been happening to me said the sad little eyes oh how sweet it was to see them snap at her again from the red eiderdown but the nose which was of some black composition wasn't at all firm it must have had a knock somehow never mind-a little dab of black sealing-wax when the time came-when it was absolutely necessary little rogue yes she really felt like that about it little rogue biting its tail just by her left ear she could have taken it off and laid it on her lap and stroked it she felt a tingling in her hands and arms but that came from walking she supposed and when she breathed something light and sad-no not sad exactly-something gentle seemed to move in her bosom there were a number of people out this afternoon far more than last sunday and the band sounded louder and gayer that was because the season had begun for although the band played all the year round on sundays out of season it was never the same it was like some one playing with only the family to listen; it didn't care how it played if there weren't any strangers present wasn't the conductor wearing a new coat too she was sure it was new he scraped with his foot and flapped his arms like a rooster about to crow and the bandsmen sitting in the green rotunda blew out their cheeks and glared at the music now there came a little flutey bit-very pretty-a little chain of bright drops she was sure it would be repeated it was; she lifted her head and smiled only two people shared her special seat: a fine old man in a velvet coat his hands clasped over a huge carved walking-stick and a big old woman sitting upright with a roll of knitting on her embroidered apron they did not speak this was disappointing for miss brill always looked forward to the conversation she had become really quite expert she thought at listening as though she didn't listen at sitting in other people's lives just for a minute while they talked round her she glanced sideways at the old couple perhaps they would go soon last sunday too hadn't been as interesting as usual an englishman and his wife he wearing a dreadful panama hat and she button boots and she'd gone on the whole time about how she ought to wear spectacles; she knew she needed them; but that it was no good getting any; they'd be sure to break and they'd never keep on and he'd been

so patient he d suggested everything-gold rims the kind that curved round your ears little pads inside the bridge no nothing would please her they ll always be sliding down my nose miss brill had wanted to shake her the old people sat on the bench still as statues never mind there was always the crowd to watch to and fro in front of the flower-beds and the band rotunda the couples and groups paraded stopped to talk to greet to buy a handful of flowers from the old beggar who had his tray fixed to the railings little children ran among them swooping and laughing; little boys with big white silk bows under their chins little girls little french dolls dressed up in velvet and lace and sometimes a tiny staggerer came suddenly rocking into the open from under the trees stopped stared as suddenly sat down flop until its small high-stepping mother like a young hen rushed scolding to its rescue other people sat on the benches and green chairs but they were nearly always the same sunday after sunday and-miss brill had often noticed-there was something funny about nearly all of them they were odd silent nearly all old and from the way they stared they looked as though they d just come from dark little rooms or even-even cupboards behind the rotunda the slender trees with yellow leaves down drooping and through them just a line of sea and beyond the blue sky with gold-veined clouds tum-tum-tum tiddle-um tiddle-um tum tiddley-um tum ta blew the band two young girls in red came by and two young soldiers in blue met them and they laughed and paired and went off arm-in-arm two peasant women with funny straw hats passed gravely leading beautiful smoke-coloured donkeys a cold pale nun hurried by a beautiful woman came along and dropped her bunch of violets and a little boy ran after to hand them to her and she took them and threw them away as if they d been poisoned dear me miss brill didn t know whether to admire that or not and now an ermine toque and a gentleman in grey met just in front of her he was tall stiff dignified and she was wearing the ermine toque she d bought when her hair was yellow now everything her hair her face even her eyes was the same colour as the shabby ermine and her hand in its cleaned glove lifted to dab her lips was a tiny yellowish paw oh she was so pleased to see him-delighted she rather thought they were going to meet that afternoon she described where she d been-everywhere here there along by the sea the day was so charming-didn t he agree and wouldn t he perhaps but he shook his head lighted a cigarette slowly breathed a great deep puff into her face and even while she was still talking and laughing flicked the match away and walked on the ermine toque was alone; she smiled more brightly than ever but even the band seemed to know what she was feeling and played more softly played tenderly and the drum beat the brute the brute over and over what would she do what was going to happen now but as miss brill wondered the ermine toque turned raised her hand as though she d seen some one else much nicer just over there and pattered away and the band changed again and played more quickly more gayly than ever and the old couple on miss brill s seat got up and marched away and such a funny old man with long whiskers hobbled along in time to the music and was nearly knocked over by four girls walking abreast oh how fascinating it was how she enjoyed it how she loved sitting here watching it all it was like a play it was exactly like a play who could believe the sky at the back wasn t painted but it wasn t till a little brown dog trotted on solemn and then slowly trotted off like a little theatre dog a little dog that had been drugged that miss brill discovered what it was that made it so

exciting they were all on the stage they weren't only the audience not only looking on; they were acting even she had a part and came every Sunday no doubt somebody would have noticed if she hadn't been there; she was part of the performance after all how strange she'd never thought of it like that before and yet it explained why she made such a point of starting from home at just the same time each week-so as not to be late for the performance-and it also explained why she had quite a queer shy feeling at telling her English pupils how she spent her Sunday afternoons no wonder Miss Brill nearly laughed out loud she was on the stage she thought of the old invalid gentleman to whom she read the newspaper four afternoons a week while he slept in the garden she had got quite used to the frail head on the cotton pillow the hollowed eyes the open mouth and the high pinched nose if he'd been dead she mightn't have noticed for weeks; she wouldn't have minded but suddenly he knew he was having the paper read to him by an actress an actress the old head lifted; two points of light quivered in the old eyes an actress-are ye and Miss Brill smoothed the newspaper as though it were the manuscript of her part and said gently; yes I have been an actress for a long time the band had been having a rest now they started again and what they played was warm sunny yet there was just a faint chill-a something what was it-not sadness-no not sadness-a something that made you want to sing the tune lifted lifted the light shone; and it seemed to Miss Brill that in another moment all of them all the whole company would begin singing the young ones the laughing ones who were moving together they would begin and the men's voices very resolute and brave would join them and then she too she too and the others on the benches-they would come in with a kind of accompaniment-something low that scarcely rose or fell something so beautiful-moving and Miss Brill's eyes filled with tears and she looked smiling at all the other members of the company yes we understand we understand she thought-though what they understood she didn't know just at that moment a boy and girl came and sat down where the old couple had been they were beautifully dressed; they were in love the hero and heroine of course just arrived from his father's yacht and still soundlessly singing still with that trembling smile Miss Brill prepared to listen no not now said the girl not here I can't but why because of that stupid old thing at the end there asked the boy why does she come here at all-who wants her why doesn't she keep her silly old mug at home it's her fu-ur which is so funny giggled the girl it's exactly like a fried whiting ah be off with you said the boy in an angry whisper then: tell me ma petite chere-no not here said the girl not yet on her way home she usually bought a slice of honey-cake at the baker's it was her Sunday treat sometimes there was an almond in her slice sometimes not it made a great difference if there was an almond it was like carrying home a tiny present-a surprise-something that might very well not have been there she hurried on the almond Sundays and struck the match for the kettle in quite a dashing way but to-day she passed the baker's by climbed the stairs went into the little dark room-her room like a cupboard-and sat down on the red eiderdown she sat there for a long time the box that the fur came out of was on the bed she unclasped the necklet quickly; quickly without looking laid it inside but when she put the lid on she thought she heard something crying

{': 40, 'faint': 2, 'gayly': 1, 'moment': 2, 'telling': 1, 'nose': 3, 'boys':

1, 'french': 1, 'took': 1, 'but': 13, 'late': 1, 'while': 3, 'so': 7, 'chins':
 1, 'met': 2, 'peasant': 1, 'beautiful': 2, 'nicer': 1, 'course': 1, 'dashing':
 1, 'rubbed': 1, 'flicked': 1, 'cleaned': 1, 'talked': 1, 'panama': 1, 'week': 1,
 'whether': 1, 'ear': 1, 'little': 19, 'chill-a': 1, 'out': 7, 'do': 1,
 'scolding': 1, 'sideways': 1, 'button': 1, 'foot': 1, 'points': 1, 'understood':
 1, 'glared': 1, 'getting': 1, 'paired': 1, 'was;': 1, 'women': 1, 'leaves': 1,
 'abreast': 1, 'shared': 1, 'away': 4, 'snap': 1, 'flower-beds': 1, 'long': 3,
 'repeated': 1, 'acting': 1, 'carved': 1, 'loud': 1, 'round': 3, 'sometimes': 3,
 'old': 14, 'quickly;': 1, 'begun': 1, 'she': 61, 'tiny': 3, 'fascinating': 1,
 'pupils': 1, 'sweet': 1, 'heard': 1, 'till': 1, 'for': 8, 'air': 1, 'everything-
 gold': 1, 'beyond': 1, 'tum-tum-tum': 1, 'turned': 1, 'stopped': 2, 'threw': 1,
 'noticed-there': 1, 'sing': 1, 'smoothed': 1, 'spots': 1, 'open': 2, 'pretty-a':
 1, 'yellow': 2, 'climbed': 1, 'green': 2, 'gentle': 1, 'something': 6,
 'sounded': 1, 'gold': 1, 'chairs': 1, 'girl': 4, 'unclasped': 1, 'too': 4,
 'suggested': 1, 'marched': 1, 'tray': 1, 'quickly': 2, 'cheeks': 1,
 'disappointing': 1, 'somebody': 1, 'hands': 2, 'put': 2, 'together': 1,
 'clasped': 1, 'must': 1, 'them;': 1, 'benches': 1, 'girls': 3, 'spectacles;': 1,
 'rose': 1, 'seat;': 1, 'when': 5, 'powdered': 1, 'from': 10, 'weren': 2,
 'would': 9, 'taken': 2, 'gently;': 1, 'into': 4, 'lap': 1, 'become': 1, 'dear':
 2, 'expert': 1, 'hats': 1, 'shabby': 1, 'minded': 1, 'spent': 1, 'this': 2,
 'queer': 1, 'laughed': 2, 'bed': 1, 'special': 1, 'went': 2, 'were': 10, 'nice':
 1, 'all-who': 1, 'keep': 2, 'having': 2, 'slice': 2, 'drifting-from': 1,
 'staggerer': 1, 'listening': 1, 'painted': 1, 'tenderly': 1, 'each': 1,
 'bridge': 1, 'fine-the': 1, 'charming-didn': 1, 'although': 2, 'gone': 1,
 'stiff': 1, 'bright': 1, 'tiddley-um': 1, 'what': 8, 'benches-they': 1,
 'passed': 2, 'shy': 1, 'admire': 1, 'number': 1, 'every': 1, 'doubt': 1,
 'fried': 1, 'brush': 1, 'paper': 1, 'buy': 1, 'there;': 1, 'singing': 2, 'on;':
 1, 'wear': 1, 'been': 11, 'whom': 1, 'loved': 1, 'dead': 1, 'petite': 1,
 'stroked': 1, 'doesn': 1, 'smiled': 2, 'still': 4, 'another': 1, 'stairs': 1,
 'tingling': 1, 'dreadful': 1, 'funny': 4, 'same': 4, 'under': 2, 'me': 3,
 'puff': 1, 'forward': 1, 'fine': 1, 'where': 2, 'about': 4, 'nearly': 5, 'was':
 45, 'soundlessly': 1, 'wondered': 1, 'been-everywhere': 1, 'tune': 1, 'velvet':
 2, 'fixed': 1, 'left': 1, 'almond': 3, 'a': 51, 'starting': 1, 'blew': 2,
 'curved': 1, 'bunch': 1, 'll': 1, 'to-day': 1, 'soon': 1, 'couples': 1,
 'stupid': 1, 'english': 1, 'low': 1, 'whiskers': 1, 'alone;': 1, 'if': 5, 'hat':
 1, 'wife': 1, 'described': 1, 'glove': 1, 'sitting': 4, 'end': 1, 'struck': 1,
 'dropped': 1, 'sat': 6, 'surprise-something': 1, 'rooms': 1, 'sliding': 1,
 'flapped': 1, 'to': 33, 'usually': 1, 'quite': 4, 'again': 5, 'black': 2,
 'sunday': 7, 'not': 12, 'brute': 2, 'talking': 1, 'dog': 3, 'read': 2,
 'composition': 1, 'which': 2, 'their': 2, 'bows': 1, 'dab': 2, 'played': 6,
 'dolls': 1, 'pinched': 1, 'hollowed': 1, 'came': 9, 'through': 1, 'lid': 1,
 'part': 3, 'accompaniment-something': 1, 'water': 1, 'gravely': 1, 'company': 2,
 'lighted': 1, 'drugged': 1, 'honey-cake': 1, 'two': 5, 'trotted': 2, 'other': 3,
 'please': 1, 'had': 14, 'crowd': 1, 'also': 1, 'manuscript': 1, 'supposed': 1,
 'firm': 1, 'ah': 1, 'usual': 1, 'actress': 3, 'rest': 1, 'newspaper': 2, 'arm-
 in-arm': 1, 'boy': 4, 'wants': 1, 'woman': 2, 'laid': 2, 'weeks;': 1, 'rims': 1,
 'tiddle-um': 2, 'flutey': 1, 'exactly-something': 1, 'toque': 4, 'difference':
 1, 'resolute': 1, 'small': 1, 'them': 10, 'wine': 1, 'swooping': 1, 'upright':

1, 'ever': 2, 'inside': 2, 'room-her': 1, 'has': 1, 'treat': 1, 'does': 1,
 'without': 1, 'face': 2, 'time': 6, 'used': 1, 'up': 3, 'gayer': 1, 'flop': 1,
 'conversation': 1, 'paw': 1, 'match': 2, 'dressed': 1, 'groups': 1, 'going': 2,
 'sadness-no': 1, 'along': 3, 'bought': 2, 'handful': 1, 'moving': 1, 'year': 1,
 'warm': 1, 'box': 2, 'love': 1, 'straw': 1, 'family': 1, 'seemed': 3, 'crow': 1,
 'solemn': 1, 'stage': 2, 'given': 1, 'giggled': 1, 'sip': 1, 'men': 1, 'happen':
 1, 'minute': 1, 'bench': 1, 'mind-a': 1, 'all': 10, 'happening': 1, 'publiques-
 miss': 1, 'way': 3, 'rooster': 1, 'among': 1, 'breathed': 2, 'ermine': 5,
 'back': 2, 'else': 1, 'silent': 1, 'trembling': 1, 'silk': 1, 'see': 2, 'room':
 1, 'the': 110, 'good': 2, 'only': 4, 'shone;': 1, 'is': 1, 'day': 1, 'bit-very':
 1, 'lifted': 4, 'high-stepping': 1, 'roll': 1, 'hurried': 2, 'after': 3,
 'lives': 1, 'cigarette': 1, 'slender': 1, 'seen': 1, 'smiling': 1, 'hand': 4,
 'enjoyed': 1, 'watch': 1, 'performance': 1, 'might': 1, 'yacht': 1, 'on': 20,
 'beat': 1, 'needed': 1, 'whiting': 1, 'shake': 1, 'railings': 1, 'because': 2,
 'scarcely': 1, 'came-when': 1, 'moth-powder': 1, 'like': 13, 'over': 6,
 'laughing;': 1, 'even': 4, 'statues': 1, 'dignified': 1, 'gold-veined': 1,
 'blue': 3, 'tum': 2, 'rotunda': 3, 's': 9, 'how': 8, 'seat': 1, 'play': 2,
 'know': 3, 'ones': 2, 'meet': 1, 'carrying': 1, 'four': 2, 'new': 2, 'colour':
 1, 'boots': 1, 'life': 1, 'louder': 1, 'laughing': 2, 'touched': 1, 'deep': 1,
 'got': 2, 'wanted': 1, 'poisoned': 1, 'front': 2, 'as': 12, 'there': 13,
 'rocking': 1, 'knock': 1, 'sea': 2, 'beautiful-moving': 1, 'dark': 2, 'listen;':
 1, 'thought-though': 1, 'silly': 1, 'line': 1, 'knocked': 1, 'brill': 15, 'now':
 7, 'great': 3, 'no': 6, 'want': 1, 'have': 7, 'discovered': 1, 'stared': 2,
 'hobbled': 1, 'd': 12, 'patient': 1, 'here': 5, 'children': 1, 'his': 7, 'far':
 1, 'then': 1, 'head': 4, 'why': 5, 'believe': 1, 'afternoons': 2, 'chere-': 1,
 'nothing': 1, 'pale': 1, 'baker': 2, 'any': 1, 'who': 3, 'him-delighted': 1,
 'walking-stick': 1, 'much': 1, 'father': 1, 'strangers': 1, 'dressed;': 1,
 'hero': 1, 'with': 13, 'than': 3, 'actress-are': 1, 'any;': 1, 'pleased': 1,
 'tail': 1, 'coat': 2, 'soldiers': 1, 'home': 4, 'jardins': 1, 'until': 1, 'red':
 3, 'clouds': 1, 'mother': 1, 'tell': 1, 'noticed': 2, 'shaken': 1, 'somehow': 1,
 'hadn': 2, 'yes': 3, 'eyes': 6, 'cotton': 1, 'voices': 1, 'rescue': 1, 'talk':
 1, 'splashed': 1, 'smoke-coloured': 1, 'very': 2, 'performance-and': 1,
 'looked': 3, 'always': 4, 'sealing-wax': 1, 'brightly': 1, 'afternoon': 3, 'oh':
 3, 'beggar': 1, 'frail': 1, 'behind': 1, 'everything': 1, 'did': 1, 'such': 2,
 'season': 2, 'never': 5, 'donkeys': 1, 'young': 4, 'brave': 1, 'sadness-a': 1,
 'fell': 1, 'eiderdown': 2, 'off': 4, 'rather': 1, 'yet': 3, 'odd': 1, 'care': 1,
 'apron': 1, 'strange': 1, 'mind': 1, 'violets': 1, 'leading': 1, 'glass': 1,
 'lips': 1, 'pattered': 1, 'ye': 1, 'her': 34, 'motionless': 1, 'embroidered': 1,
 'fro': 1, 'often': 1, 'dim': 1, 'week-so': 1, 'kettle': 1, 'wearing': 3, 'its':
 5, 'that': 21, 'wasn': 4, 'hair': 2, 'necessary': 1, 'ears': 1, 'even-even': 1,
 'just': 11, 'rushed': 1, 'iced': 1, 'others': 1, 'sad': 2, 'brilliantly': 1,
 'ta': 1, 'cold': 1, 'prepared': 1, 'arms': 2, 'suddenly': 3, 'raised': 1, 'fu-
 ur': 1, 'really': 2, 'your': 2, 'last': 2, 'cupboards': 1, 'join': 1, 'begin':
 2, 'understand': 2, 'trees': 2, 'bandsmen': 1, 'hen': 1, 'paraded': 1, 'tall':
 1, 'theatre': 1, 'sunny': 1, 'then': 2, 'didn': 4, 'feeling': 2, 'mug': 1,
 'break': 1, 'softly': 1, 'decided': 1, 'at': 15, 'before': 2, 'crying': 1,
 'listen': 2, 'nun': 1, 'or': 3, 'huge': 1, 'slept': 1, 'arrived': 1, 'looking':
 2, 'cupboard-and': 1, 'interesting': 1, 'quivered': 1, 'can': 1, 'more': 5,


```
'members': 1, 'could': 2, 'necklet': 1, 'go': 1, 'lifted;': 1, 'smile': 1,
'and': 77, 't': 18, 'people': 5, 'we': 2, 'present': 1, 'knitting': 1, 'light':
4, 'beautifully': 1, 'gentleman': 2, 'lace': 1, 'high': 1, 'white': 2,
'exactly': 2, 'nowhere': 1, 'miss': 13, 'you': 4, 'wouldn': 2, 'speak': 1,
'invalid': 1, 'him': 1, 'bosom': 1, 'man': 2, 'sky': 4, 'in': 21, 'drooping': 1,
'yellowish': 1, 'ran': 2, 'couple': 3, 'present-a': 1, 'walking': 2, 'chill': 2,
'thing': 2, 'changed': 1, 'drum': 1, 'down': 5, 'heroine': 1, 'come': 3, 'feel':
1, 'kind': 2, 'angry': 1, 'big': 2, 'an': 9, 'watching': 1, 'it-not': 1,
'biting': 1, 'some': 3, 'brown': 1, 'filled': 1, 'point': 1, 'asked': 1, 'well':
1, 'tears': 1, 'fur': 3, 'of': 28, 'glanced': 1, 'felt': 2, 'it': 39, 'playing':
1, 'he': 12, 'walked': 1, 'ought': 1, 'agree': 1, 'started': 1, 'glad': 1,
'thought': 5, 'sad-no': 1, 'said': 5, 'move': 1, 'drops': 1, 'rogue': 2, 'be':
5, 'wonder': 1, 'explained': 2, 'music': 2, 'chain': 1, 'mightn': 1, 'one': 2,
'slowly': 2, 'mouth': 2, 'leaf': 1, 'scraped': 1, 'pads': 1, 'audience': 1,
'sundays': 2, 'my': 1, 'sure': 3, 'greet': 1, 'shook': 1, 'exciting': 1,
'opened': 1, 'band': 7, 'though': 4, 'perhaps': 2, 'by': 7, 'whole': 2,
'flowers': 1, 'whisper': 1, 'garden': 1, 'grey': 1, 'and-miss': 1, 'englishman':
1, 'knew': 2, 'made': 4, 'ma': 1, 'they': 23, 'pillow': 1, 'i': 2, 'absolutely':
1, 'conductor': 1}
```

The word ermine appears 5 times.

1.6 Module 4 - Working with Data in Python

1.6.1 Reading Files with Open

Reading files

Create Example1.txt Create a file named 'Example1.txt' with following content: - This is line 1 - This is line 2 - This is line 3

```
[158]: with open('Example1.txt', 'w+') as example:
        example.write('This is line 1\n')
        example.write('This is line 2\n')
        example.write('This is line 3')
```

```
[159]: file1 = open('Example1.txt', 'r')
        print(file1.name)
        print(file1.mode)
        file1Content = file1.read()
        print(file1Content)
        print(type(file1Content))
        print(file1.closed)
        file1.close()
        print(file1.closed)
```

```
Example1.txt
r
This is line 1
This is line 2
```

```
This is line 3
<class 'str'>
False
True
```

```
[160]: with open('Example1.txt', 'r') as file1:
        file1Content = file1.read()
        print(file1Content)
        print(file1.closed)
        print(file1.closed)
```

```
This is line 1
This is line 2
This is line 3
False
True
```

```
[161]: with open('Example1.txt', 'r') as file1:
        print(file1.read())
        print(file1.closed)
        print(file1.closed)
```

```
This is line 1
This is line 2
This is line 3
False
True
```

```
[162]: with open('Example1.txt', 'r') as file1:
        print(file1.read(4))
        print(file1.read(4))
        print(file1.read(9))
        print(file1.closed)
        print(file1.closed)
```

```
This
is
line 1
Th
False
True
```

```
[163]: with open('Example1.txt', 'r') as file1:
        print(file1.readline())
        print(file1.readline())
        print(file1.readline(4))
        print(file1.closed)
        print(file1.closed)
```

This is line 1

This is line 2

This
False
True

```
[164]: with open('Example1.txt', 'r') as file1:
        i = 0;
        for line in file1:
            print('Iteration', str(i), ': ', line)
            i = i + 1
```

Iteration 0 : This is line 1

Iteration 1 : This is line 2

Iteration 2 : This is line 3

```
[165]: with open('Example1.txt', 'r') as file1:
        FileasList = file1.readlines()
        print(FileasList)
        print(FileasList[1])
```

['This is line 1\n', 'This is line 2\n', 'This is line 3']
This is line 2

Writing files

```
[166]: with open('Example2.txt', 'w') as file2:
        file2.write('This is line A')
        with open('Example2.txt', 'r') as file2:
            print(file2.read())
```

This is line A

```
[167]: with open('Example2.txt', 'w') as file2:
        file2.write('This is line A')
        file2.write('This is line B')
        file2.write('This is line C')
        with open('Example2.txt', 'r') as file2:
            print(file2.read())
```

This is line AThis is line BThis is line C

```
[168]: with open('Example2.txt', 'w') as file2:
        file2.write('This is line A\n')
        file2.write('This is line B\n')
        file2.write('This is line C')
    with open('Example2.txt', 'r') as file2:
        print(file2.read())
```

```
This is line A
This is line B
This is line C
```

```
[169]: with open('Example2.txt', 'a') as file2:
        file2.write('\n')
        file2.write('This is line D\n')
        file2.write('This is line E\n')
        file2.write('This is line F')
    with open('Example2.txt', 'r') as file2:
        print(file2.read())
```

```
This is line A
This is line B
This is line C
This is line D
This is line E
This is line F
```

Additional modes It's fairly inefficient to open the file in **a** or **w** and then reopen it in **r** to read any lines. Luckily we can access the file in the following modes: - **r+** : Reading and writing. Cannot truncate the file. - **w+** : Writing and reading. Truncates the file. - **a+** : Appending and Reading. Creates a new file, if none exists.

It is often very useful to know where the 'cursor' is in a file and be able to control it. The following methods allow us to do precisely this - **.tell()** - returns the current position in bytes - **.seek(offset,from)** - changes the position by 'offset' bytes with respect to 'from'. From can take the value of 0,1,2 corresponding to the beginning, relative to current position and end

Finally, a note on the difference between **w+** and **r+**. Both of these modes allow access to read and write methods; however, opening a file in **w+** overwrites it and deletes all pre-existing data. To work with a file on existing data, use **r+** and **a+**. While using **r+**, it can be useful to add a **.truncate()** method at the end of your data. This will reduce the file to your data and delete everything that follows. In the following code block, Run the code as it is first and then run it with the **.truncate()**.

```
[170]: with open('Example2.txt', 'a+') as file2:
        file2.write('\n')
        file2.write('This is line G')
        print('Current location: {}'.format(file2.tell()))
        print(file2.read())
```

```

file2.seek(0,0)
print('Current location: {}'.format(file2.tell()))
print(file2.read())

```

Current location: 104

Current location: 0
This is line A
This is line B
This is line C
This is line D
This is line E
This is line F
This is line G

```

[171]: with open('Example2.txt', 'r') as file2:
        with open('Example3.txt', 'w') as file3:
            for line in file2:
                file3.write(line)
with open('Example3.txt', 'r') as file3:
    print(file3.read())

```

This is line A
This is line B
This is line C
This is line D
This is line E
This is line F
This is line G

Exercise Your local university’s Raptors fan club maintains a register of its active members on a .txt document. Every month they update the file by removing the members who are not active. You have been tasked with automating this with your Python skills. Given the file `currentMem`, Remove each member with a ‘no’ in their Active column. Keep track of each of the removed members and append them to the `exMem` file. Make sure that the format of the original files is preserved. (*Hint: Do this by reading/writing whole lines and ensuring the header remains*) Run the code block below prior to starting the exercise. The skeleton code has been provided for you. Edit only the `cleanFiles` function.

```

[172]: # Run this prior to starting the exercise
from random import randint as rnd
memReg = 'members.txt'
exReg = 'inactive.txt'
fee = ('yes','no')
def genFiles(current,old):
    with open(current,'w+') as writefile:
        writefile.write('Membership No  Date Joined  Active  \n')

```

```

data = "{:~13} {:<11} {:<6}\n"
for rowno in range(20):
    date = str(rnd(2015,2020))+ '-' + str(rnd(1,12))+ '-' + str(rnd(1,25))
    writefile.write(data.format(rnd(10000,99999),date,fee[rnd(0,1)]))
with open(old,'w+') as writefile:
    writefile.write('Membership No Date Joined Active \n')
    data = "{:~13} {:<11} {:<6}\n"
    for rowno in range(3):
        date = str(rnd(2015,2020))+ '-' + str(rnd(1,12))+ '-' + str(rnd(1,25))
        writefile.write(data.format(rnd(10000,99999),date,fee[1]))
genFiles(memReg,exReg)

```

```

[173]: def cleanFiles(currentMem,exMem):
    with open(currentMem,'r+') as writeFile:
        with open(exMem,'a+') as appendFile:
            # Get the data
            writeFile.seek(0)
            members = writeFile.readlines()
            # Remove header
            header = members[0]
            members.pop(0)

            inactive = [member for member in members if ('no' in member)]
            '''
            The above is the same as

            for member in members:
                if 'no' in member:
                    inactive.append(member)
            '''

            # Go to the beginning of the write file
            writeFile.seek(0)
            writeFile.write(header)
            for member in members:
                if (member in inactive):
                    appendFile.write(member)
                else:
                    writeFile.write(member)
            writeFile.truncate()

memReg = 'members.txt'
exReg = 'inactive.txt'
cleanFiles(memReg,exReg)

```

1.6.2 Pandas

```
[174]: import pandas as pd
```

```
[175]: x = {'Name': ['Rose', 'John', 'Jane', 'Mary'],
          'ID': [1, 2, 3, 4],
          'Department': ['Architect Group', 'Software Group', 'Design Team',
          ↪ 'Infrastructure'],
          'Salary': [100000, 80000, 50000, 60000]}

# Casting the dictionary to a DataFrame
df = pd.DataFrame(x)

# Display the result df
print(df)
df
```

	Name	ID	Department	Salary
0	Rose	1	Architect Group	100000
1	John	2	Software Group	80000
2	Jane	3	Design Team	50000
3	Mary	4	Infrastructure	60000

```
[175]:
```

	Name	ID	Department	Salary
0	Rose	1	Architect Group	100000
1	John	2	Software Group	80000
2	Jane	3	Design Team	50000
3	Mary	4	Infrastructure	60000

Exercise 1

```
[176]: # Defining the structure of the dictionary using indentations for better
      ↪ readability
data = {'': [ '', '', '', '' ],
        '': [ '', '', '', '' ],
        '': [ '', '', '', '' ],
        '': [ '', '', '', '' ]}
df1 = pd.DataFrame(data)
print(df1)

# Populating the data into the dictionary
data = {'Student': [ 'David', 'Samuel', 'Terry', 'Evan'],
        'Age': [ 27, 24, 22, 32],
        'Country': [ 'UK', 'Canada', 'China', 'USA'],
        'Course': [ 'Python', 'Data Structures', 'Machine Learning', 'Web
        ↪ Development'],
        'Marks': [ 85, 72, 89, 76]}
df1 = pd.DataFrame(data)
print(df1)
```

```
print(type(df1))
df1
```

```
0
1
2
3
  Student  Age Country      Course  Marks
0   David   27     UK        Python    85
1  Samuel   24  Canada  Data Structures    72
2   Terry   22   China  Machine Learning    89
3    Evan   32     USA  Web Development    76
<class 'pandas.core.frame.DataFrame'>
```

```
[176]:   Student  Age Country      Course  Marks
0   David   27     UK        Python    85
1  Samuel   24  Canada  Data Structures    72
2   Terry   22   China  Machine Learning    89
3    Evan   32     USA  Web Development    76
```

```
[177]: print(df1[['Age']])
print(df1[['Age']].mean())
```

```
      Age
0      27
1      24
2      22
3      32
Age      26.25
dtype: float64
```

```
[178]: print(df1[['Marks']])
print(df1[['Marks']].mean())
```

```
      Marks
0        85
1        72
2        89
3        76
Marks      80.5
dtype: float64
```

```
[179]: a = df1[['Country', 'Course']]
print(a)
print(type(a))
a
```



```

      Country      Course
0      UK      Python
1  Canada  Data Structures
2   China  Machine Learning
3    USA   Web Development
<class 'pandas.core.frame.DataFrame'>

```

```

[179]:      Country      Course
0      UK      Python
1  Canada  Data Structures
2   China  Machine Learning
3    USA   Web Development

```

```

[180]: # Dataframes require the indexes to be in square brackets
b = df1[['Student']]
print(b)
print(type(b))
b

```

```

      Student
0    David
1   Samuel
2    Terry
3     Evan
<class 'pandas.core.frame.DataFrame'>

```

```

[180]:      Student
0    David
1   Samuel
2    Terry
3     Evan

```

```

[181]: # Series require the index not to be in square brackets
b = df1['Student']
print(b)
print(type(b))
b

```

```

0    David
1   Samuel
2    Terry
3     Evan
Name: Student, dtype: object
<class 'pandas.core.series.Series'>

```

```

[181]: 0    David
1    Samuel
2    Terry

```

```
3      Evan
Name: Student, dtype: object
```

Exercise 2

```
[182]: df
```

```
[182]:
```

	Name	ID	Department	Salary
0	Rose	1	Architect Group	100000
1	John	2	Software Group	80000
2	Jane	3	Design Team	50000
3	Mary	4	Infrastructure	60000

```
[183]: print(df[['Salary']])
print(df[['Salary']].mean())
```

```
Salary
0  100000
1   80000
2   50000
3   60000
Salary    72500.0
dtype: float64
```

```
[184]: print(df.iloc[0, 0])
df.iloc[0, 0]
```

```
Rose
```

```
[184]: 'Rose'
```

```
[185]: print(df.iloc[0, 2])
df.iloc[0, 2]
```

```
Architect Group
```

```
[185]: 'Architect Group'
```

```
[186]: print(df.loc[0, 'Salary'])
```

```
100000
```

```
[187]: df2 = df
df2 = df2.set_index('Name')
print(df2.head())
df2.head()
```

```
      ID      Department  Salary
Name
```

Rose	1	Architect Group	100000
John	2	Software Group	80000
Jane	3	Design Team	50000
Mary	4	Infrastructure	60000

```
[187]:      ID      Department  Salary
      Name
Rose    1  Architect Group  100000
John    2   Software Group   80000
Jane    3     Design Team   50000
Mary    4  Infrastructure   60000
```

```
[188]: print(df2.loc['John', 'Department'])
      df2.loc['John', 'Department']
```

Software Group

```
[188]: 'Software Group'
```

```
[189]: print(df2.loc['Jane', 'Salary'])
      df2.loc['Jane', 'Salary']
```

50000

```
[189]: 50000
```

Exercise 3

```
[190]: df
```

```
[190]:      Name  ID      Department  Salary
0  Rose    1  Architect Group  100000
1  John    2   Software Group   80000
2  Jane    3     Design Team   50000
3  Mary    4  Infrastructure   60000
```

```
[191]: print(df.iloc[0:2, 0:3])
      df.iloc[0:2, 0:3]
```

	Name	ID	Department
0	Rose	1	Architect Group
1	John	2	Software Group

```
[191]:      Name  ID      Department
0  Rose    1  Architect Group
1  John    2   Software Group
```

```
[192]: print(df.loc[0:2, 'ID':'Department'])
df.loc[0:2, 'ID':'Department']
```

	ID	Department
0	1	Architect Group
1	2	Software Group
2	3	Design Team

```
[192]: ID      Department
0     1  Architect Group
1     2   Software Group
2     3    Design Team
```

```
[193]: df2
```

```
[193]: ID      Department  Salary
Name
Rose   1  Architect Group  100000
John   2   Software Group   80000
Jane   3    Design Team    50000
Mary   4  Infrastructure    60000
```

```
[194]: print(df2.loc['Rose':'Jane', 'ID':'Department'])
df2.loc['Rose':'Jane', 'ID':'Department']
```

	ID	Department
Rose	1	Architect Group
John	2	Software Group
Jane	3	Design Team

```
[194]: ID      Department
Name
Rose   1  Architect Group
John   2   Software Group
Jane   3    Design Team
```

```
[195]: df
```

```
[195]: Name  ID      Department  Salary
0  Rose   1  Architect Group  100000
1  John   2   Software Group   80000
2  Jane   3    Design Team    50000
3  Mary   4  Infrastructure    60000
```

```
[196]: print(df.loc[2:3, 'Name':'Department'])
df.loc[2:3, 'Name':'Department']
```

	Name	ID	Department
2	Jane	3	Design Team
3	Mary	4	Infrastructure

```
[196]:
```

	Name	ID	Department
2	Jane	3	Design Team
3	Mary	4	Infrastructure

Exercise 4

Create TopSellingAlbums.csv Create a file named ‘TopSellingAlbums.csv’ with following content: Artist,Album,Released,Length,Genre,Music Recording Sales (millions),Claimed Sales (millions),Released,Soundtrack,Rating Michael Jackson,Thriller,1982,0:42:19,“pop, rock, R&B”,46,65,30-Nov-82,,10 AC/DC,Back in Black,1980,0:42:11,hard rock,26.1,50,25-Jul-80,,9.5 Pink Floyd,The Dark Side of the Moon,1973,0:42:49,progressive rock,24.2,45,01-Mar-73,,9 Whitney Houston,The Bodyguard,1992,0:57:44,“R&B, soul, pop”,27.4,44,17-Nov-92,Y,8.5 Meat Loaf,Bat Out of Hell,1977,0:46:33,“hard rock, progressive rock”,20.6,43,21-Oct-77,,8 Eagles,Their Greatest Hits (1971-1975),1976,0:43:08,“rock, soft rock, folk rock”,32.2,42,17-Feb-76,,7.5 Bee Gees,Saturday Night Fever,1977,1:15:54,disco,20.6,40,15-Nov-77,Y,7 Fleetwood Mac,Rumours,1977,0:40:01,soft rock,27.9,40,04-Feb-77,,6.5

```
[197]: with open('TopSellingAlbums.csv', 'w+') as albums:
    albums.write('Artist,Album,Released,Length,Genre,Music Recording Sales,
    Claimed Sales (millions),Released,Soundtrack,Rating\n')
    albums.write('Michael Jackson,Thriller,1982,0:42:19,"pop, rock,
    R&B",46,65,30-Nov-82,,10\n')
    albums.write('AC/DC,Back in Black,1980,0:42:11,hard rock,26.
    1,50,25-Jul-80,,9.5\n')
    albums.write('Pink Floyd,The Dark Side of the Moon,1973,0:42:49,progressive
    rock,24.2,45,01-Mar-73,,9\n')
    albums.write('Whitney Houston,The Bodyguard,1992,0:57:44,"R&B, soul,
    pop",27.4,44,17-Nov-92,Y,8.5\n')
    albums.write('Meat Loaf,Bat Out of Hell,1977,0:46:33,"hard rock,
    progressive rock",20.6,43,21-Oct-77,,8\n')
    albums.write('Eagles,Their Greatest Hits (1971-1975),1976,0:43:08,"rock,
    soft rock, folk rock",32.2,42,17-Feb-76,,7.5\n')
    albums.write('Bee Gees,Saturday Night Fever,1977,1:15:54,disco,20.
    6,40,15-Nov-77,Y,7\n')
    albums.write('Fleetwood Mac,Rumours,1977,0:40:01,soft rock,27.
    9,40,04-Feb-77,,6.5')
```

```
[198]: df3 = pd.read_csv('TopSellingAlbums.csv', header = 0)
df3.head()
```

```
[198]:
```

	Artist	Album	Released	Length	\
0	Michael Jackson	Thriller	1982	0:42:19	

1	AC/DC	Back in Black	1980	0:42:11
2	Pink Floyd	The Dark Side of the Moon	1973	0:42:49
3	Whitney Houston	The Bodyguard	1992	0:57:44
4	Meat Loaf	Bat Out of Hell	1977	0:46:33

	Genre	Music Recording Sales (millions)	\
0	pop, rock, R&B	46.0	
1	hard rock	26.1	
2	progressive rock	24.2	
3	R&B, soul, pop	27.4	
4	hard rock, progressive rock	20.6	

	Claimed Sales (millions)	Released.1	Soundtrack	Rating
0	65	30-Nov-82	NaN	10.0
1	50	25-Jul-80	NaN	9.5
2	45	01-Mar-73	NaN	9.0
3	44	17-Nov-92	Y	8.5
4	43	21-Oct-77	NaN	8.0

```
[199]: length = df3[['Length']]
print(length)
print(type(length))
length
```

```

      Length
0  0:42:19
1  0:42:11
2  0:42:49
3  0:57:44
4  0:46:33
5  0:43:08
6  1:15:54
7  0:40:01
<class 'pandas.core.frame.DataFrame'>
```

```
[199]:      Length
0  0:42:19
1  0:42:11
2  0:42:49
3  0:57:44
4  0:46:33
5  0:43:08
6  1:15:54
7  0:40:01
```

```
[200]: art_len_gen = df3[['Artist', 'Length', 'Genre']]
print(art_len_gen)
```

```
print(type(art_len_gen))
art_len_gen
```

	Artist	Length	Genre
0	Michael Jackson	0:42:19	pop, rock, R&B
1	AC/DC	0:42:11	hard rock
2	Pink Floyd	0:42:49	progressive rock
3	Whitney Houston	0:57:44	R&B, soul, pop
4	Meat Loaf	0:46:33	hard rock, progressive rock
5	Eagles	0:43:08	rock, soft rock, folk rock
6	Bee Gees	1:15:54	disco
7	Fleetwood Mac	0:40:01	soft rock

```
<class 'pandas.core.frame.DataFrame'>
```

```
[200]:
```

	Artist	Length	Genre
0	Michael Jackson	0:42:19	pop, rock, R&B
1	AC/DC	0:42:11	hard rock
2	Pink Floyd	0:42:49	progressive rock
3	Whitney Houston	0:57:44	R&B, soul, pop
4	Meat Loaf	0:46:33	hard rock, progressive rock
5	Eagles	0:43:08	rock, soft rock, folk rock
6	Bee Gees	1:15:54	disco
7	Fleetwood Mac	0:40:01	soft rock

```
[201]: print(df3.loc[1:4, 'Album'])
df3.loc[1:4, 'Album']
```

```
1          Back in Black
2  The Dark Side of the Moon
3          The Bodyguard
4          Bat Out of Hell
Name: Album, dtype: object
```

```
[201]: 1          Back in Black
2  The Dark Side of the Moon
3          The Bodyguard
4          Bat Out of Hell
Name: Album, dtype: object
```

```
[202]: print(df3.iloc[1:5, 1])
df3.iloc[1:5, 1]
```

```
1          Back in Black
2  The Dark Side of the Moon
3          The Bodyguard
4          Bat Out of Hell
Name: Album, dtype: object
```

```
[202]: 1          Back in Black
      2  The Dark Side of the Moon
      3          The Bodyguard
      4          Bat Out of Hell
      Name: Album, dtype: object
```

1.6.3 Numpy in Python

- Numpy is a basis for Pandas

```
[203]: import numpy as np
```

```
[204]: print(np.__version__)
```

1.24.3

```
[205]: a = np.array([0, 1, 2, 3, 4, 5])
      print(a)
      print(type(a))
      print(a.dtype)
```

```
[0 1 2 3 4 5]
<class 'numpy.ndarray'>
int64
```

```
[206]: b = np.array([0, 1.1, 2, 3, 4, 5])
      print(b)
      print(b.dtype)
```

```
[0.  1.1 2.  3.  4.  5. ]
float64
```

```
[207]: c = b[3:6]
      print(c)
      print(c.dtype)
```

```
[3.  4.  5.]
float64
```

```
[208]: arr = np.array([1, 2, 3, 4, 5, 6, 7, 8])
      print(arr[0::2])
```

```
[1 3 5 7]
```

Adding arrays and plotting

```
[209]: import matplotlib.pyplot as plt
      %matplotlib inline
      def plot_vec1(x, y, z):
```

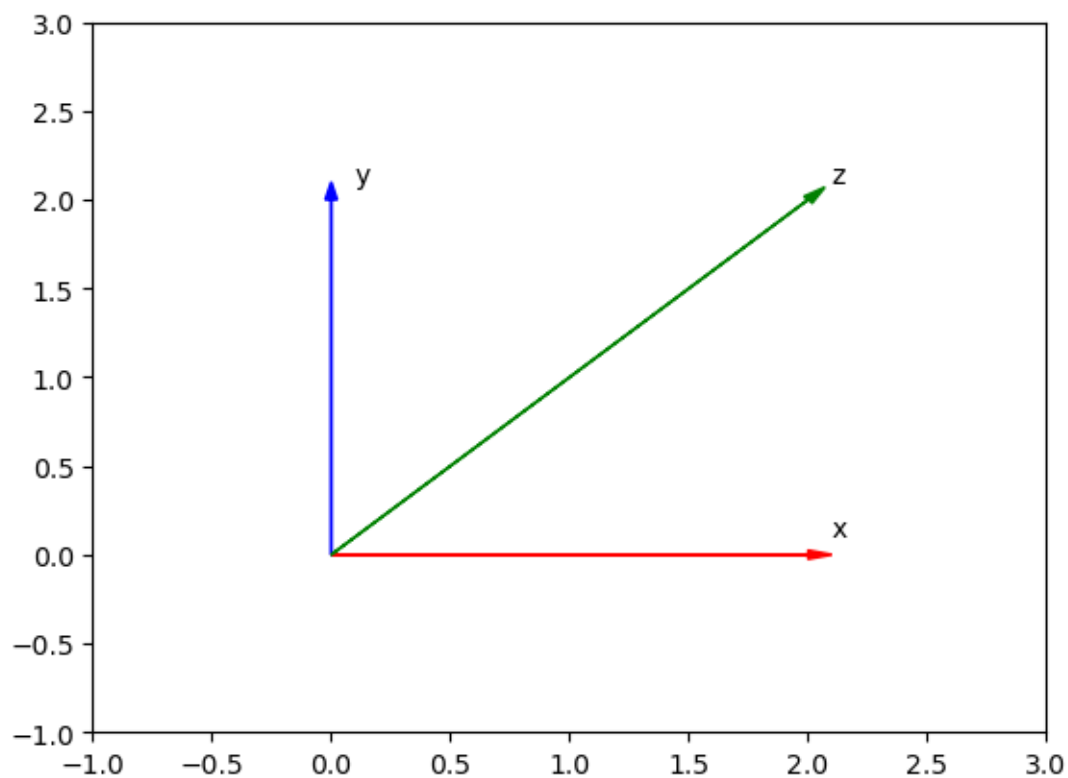


```

ax = plt.axes() # Generates the full window axes
ax.arrow(0, 0, *x, head_width = 0.05, color = 'red', head_length = 0.1) #
↳ Adds an arrow to the x axis
plt.text(*(x + 0.1), 'x') # Adds the text x to the axis
ax.arrow(0, 0, *y, head_width = 0.05, color = 'blue', head_length = 0.1) #
↳ Adds an arrow to the y axis
plt.text(*(y + 0.1), 'y') # Adds the text y to the axis
ax.arrow(0, 0, *z, head_width = 0.05, color = 'green', head_length = 0.1) #
↳ Adds an arrow to the z axis
plt.text(*(z + 0.1), 'z') # Adds the text z to the axis
plt.ylim(-1, 3) # Sets the ylim to bottom(-2), top(2)
plt.xlim(-1, 3) # Sets the xlim to left(-2), right(2)
x = np.array([2, 0])
y = np.array([0, 2])
z = np.add(x, y)
print(z)
plot_vec1(x, y, z)

```

[2 2]



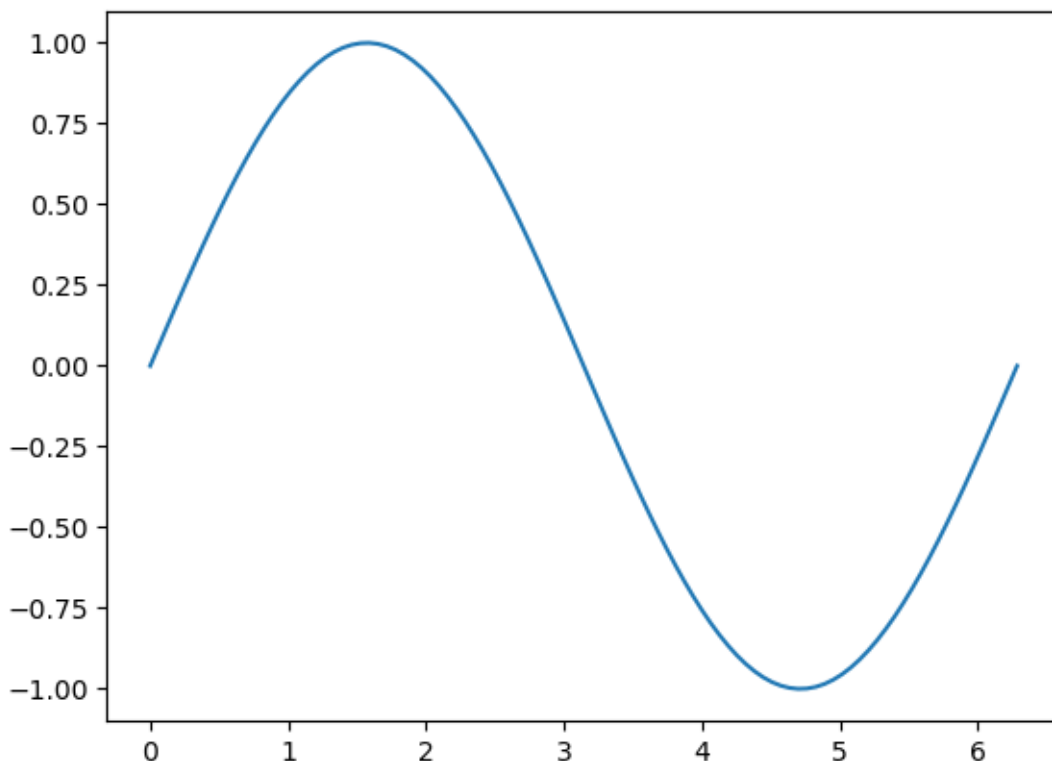
Plotting with linspace

```
[210]: x = np.linspace(0, 2*np.pi, num = 100)
print(x)
```

```
[0.          0.06346652 0.12693304 0.19039955 0.25386607 0.31733259
0.38079911 0.44426563 0.50773215 0.57119866 0.63466518 0.6981317
0.76159822 0.82506474 0.88853126 0.95199777 1.01546429 1.07893081
1.14239733 1.20586385 1.26933037 1.33279688 1.3962634  1.45972992
1.52319644 1.58666296 1.65012947 1.71359599 1.77706251 1.84052903
1.90399555 1.96746207 2.03092858 2.0943951  2.15786162 2.22132814
2.28479466 2.34826118 2.41172769 2.47519421 2.53866073 2.60212725
2.66559377 2.72906028 2.7925268  2.85599332 2.91945984 2.98292636
3.04639288 3.10985939 3.17332591 3.23679243 3.30025895 3.36372547
3.42719199 3.4906585  3.55412502 3.61759154 3.68105806 3.74452458
3.8079911  3.87145761 3.93492413 3.99839065 4.06185717 4.12532369
4.1887902  4.25225672 4.31572324 4.37918976 4.44265628 4.5061228
4.56958931 4.63305583 4.69652235 4.75998887 4.82345539 4.88692191
4.95038842 5.01385494 5.07732146 5.14078798 5.2042545  5.26772102
5.33118753 5.39465405 5.45812057 5.52158709 5.58505361 5.64852012
5.71198664 5.77545316 5.83891968 5.9023862  5.96585272 6.02931923
6.09278575 6.15625227 6.21971879 6.28318531]
```

```
[211]: y = np.sin(x)
plt.plot(x, y)
```

```
[211]: [<matplotlib.lines.Line2D at 0x7f567064f150>]
```



Finding the even and odd elements

```
[212]: # Programme to list the even and odd elements in two arrays
arr1 = np.array([1, 2, 3, 4, 5])
arr2 = np.array([6, 7, 8, 9, 10])
even = []
odd = []
for element in arr1:
    if element % 2 == 0:
        even.append(element)
    else:
        odd.append(element)
for element in arr2:
    if element % 2 == 0:
        even.append(element)
    else:
        odd.append(element)
print('Even elements:', even)
print(type(even))
print('Odd elements:', odd)
print(type(odd))
```

```
Even elements: [2, 4, 6, 8, 10]
<class 'list'>
Odd elements: [1, 3, 5, 7, 9]
<class 'list'>
```

2D Numpy

```
[213]: # The dot product requires that the quantity of columns of the multiplicand
↪ equals the quantity of rows of the multiplier. The resulting product will
↪ have the quantity of rows of the multiplicand and the quantity of columns of
↪ the multiplicand
array_a = np.array([[34, 56, 78, 42, 3, 1],
                    [84, 233, 53, 2, 8, 2],
                    [87, 36, 66, 23, 6, 8],
                    [3, 67, 226, 453, 45, 74],
                    [38, 94, 2, 0, 26, 92]])

print(array_a)
print(type(array_a))
print(array_a.dtype)
print(array_a.ndim)
print(array_a.shape)
print(array_a.size)
array_b = np.array([[59, 23, 7, 2, 0],
                    [368, 12, 42, 32, 64],
```

```

[92, 46, 2, 6, 26],
[17, 32, 69, 22, 68],
[74, 852, 89, 28, 56],
[49, 52, 94, 83, 28]])

print(array_b)
print(type(array_b))
print(array_b.dtype)
print(array_b.ndim)
print(array_b.shape)
print(array_b.size)
array_c = np.dot(array_a, array_b)
print(array_c)
print(type(array_c))
print(array_c.dtype)
print(array_c.ndim)
print(array_c.shape)
print(array_c.size)
array_d = array_c.T
print(array_d)
print(type(array_d))
print(array_d.dtype)
print(array_d.ndim)
print(array_d.shape)
print(array_d.size)

```

```

[[ 34  56  78  42   3   1]
 [ 84 233  53   2   8   2]
 [ 87  36  66  23   6   8]
 [   3  67 226 453  45  74]
 [ 38  94   2   0  26  92]]
<class 'numpy.ndarray'>
int64
2
(5, 6)
30
[[ 59  23   7   2   0]
 [368  12  42  32  64]
 [ 92  46   2   6  26]
 [ 17  32  69  22  68]
 [ 74 852  89  28  56]
 [ 49  52  94  83  28]]
<class 'numpy.ndarray'>
int64
2
(6, 5)
30
[[30775  8994  6005  3419  8664]

```

```

[96300 14150 11518 8376 16930]
[25680 11733 5126 3060 6144]
[60282 67953 45505 20874 45560]
[43450 29030 15180 11460 10100]]
<class 'numpy.ndarray'>
int64
2
(5, 5)
25
[[30775 96300 25680 60282 43450]
 [ 8994 14150 11733 67953 29030]
 [ 6005 11518 5126 45505 15180]
 [ 3419 8376 3060 20874 11460]
 [ 8664 16930 6144 45560 10100]]
<class 'numpy.ndarray'>
int64
2
(5, 5)
25

```

1.7 Module 5 - APIs and Data Collection

1.7.1 Simple APIs

[]:

1.7.2 REST APIs, Webscraping and Working with Files

[]: